

循环神经网络

前面我们处理了两个类型的数据：表格（数组）和图片，分别对应多层感知机和卷积神经网络。

实际上，现实中很多数据是**有顺序的**，例如气温，股票价格，语言生成等。为了处理这些和时间、顺序相关的数据，我们做以下探索。

例子：股票价格预测

假设我们知道 t 个离散时间点上的股票价格，分别记为 $x_1, x_2 \dots x_t$ ，现在我们要预测 x_{t+1}

最自然的想法： x_{t+1} 和前面的 t 个数据都相关，通过分析前面的所有数据来预测将来数据

困难：随着 t 的变化，输入数量也变化，不方便训练模型

解决 1（自回归模型）：只看最近 τ 时间内的数据（即 $x_{t-\tau}, x_{t-\tau+1} \dots x_{t-1}$ ）来预测，这样输入数固定

解决 2（因变量自回归模型）：既然更久远的数据数量会随着时间的变化而变化，我们把这部分数据总结成一个能代表它们的值，和更近的数据一起来预测就好了

如下图所示，隐状态 h 就是上面说的代表值。

一次输入-预测流程（中间一列）：输入了 x_{t-1} ，根据 h_{t-1} ，更新得到当前隐状态

$$h_t = g(h_{t-1}, x_{t-1})$$

然后用当前隐状态 h_t 去预测当前值：

$$\hat{x}_t = P(x_t | h_t)$$

所以图里不是直接用 x_{t-1} 预测 x_t ，而是：

$$x_{t-1} + h_{t-1} \rightarrow h_t \rightarrow \hat{x}_t$$

其中 h_{t-1} 已经包含了更早的历史信息。

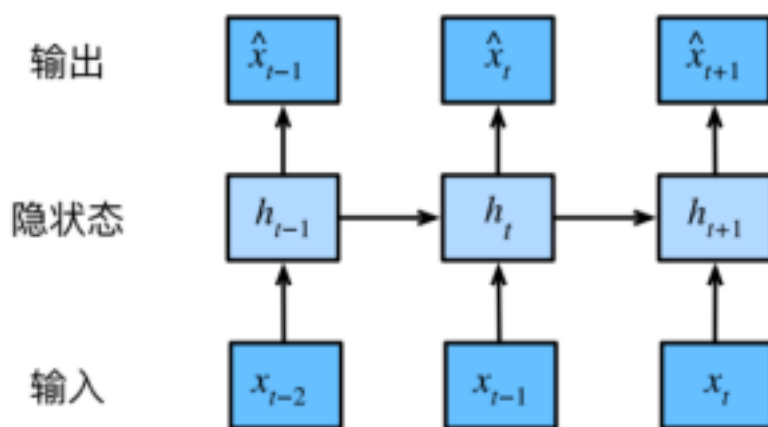


图8.1.2: 隐变量自回归模型

上面给的例子是**一步预测**（要预测的值的的时间只在已知范围的一步之后）。与之相对的是**多步预测**（例如，预测 x_{t+5} ）。此时，按照最自然的想法，该数据和其前面所有数据都关联，则需要先预测其前面的数据（ $x_t, x_{t+1} \dots x_{t+4}$ ），然后根据已知数据以及**这些预测值**来预测更远的 x_{t+5} 。这就导致偏差在预测中不断放大。显然，多步预测的准确度随着步数的增大会大幅下降。

下面介绍一些自然语言处理和语言模型中的内容

文本预处理

顾名思义，就是把自然语言按照一定的方法转化为数字集合，这样才能作为模型的输入进行训练。具体有以下三步：

1. 拿到的文本，作为一个字符串，将该字符串拆分为词元（token）这一步叫做 tokenization（一般来说有两种拆解方式：按照词拆和按照字符拆，这里选择按照词拆）
2. 构建一个词表：将上面拆解得到的词映射到独一无二的数字上去。除了这些得到的词以外，还有一些特殊的标记字符等也要映射为数字（例如未知词元 `<unk>` 等）
3. 根据词表，将原文本转化为一个数组方便模型操作

语言模型和数据集

上面完成了从文本到数字的转变，但这离我们的终极目标——预测下一个词是什么还很远。

这一点和前面讲的股票很像，本质上就是知道了按顺序排列的 n 个数据后的数据是多少。

但是**存在致命问题**：很多合理的词组在上面的语料库中没有出现过，这就必然导致预测下一个词为这个词的概率为 0，这显然不合理

三个小知识：

n 元语法：就是认为某一个词的出现只和最近的 n 个词有关

下面的表达式分别就是 1 元, 2 元和 3 元

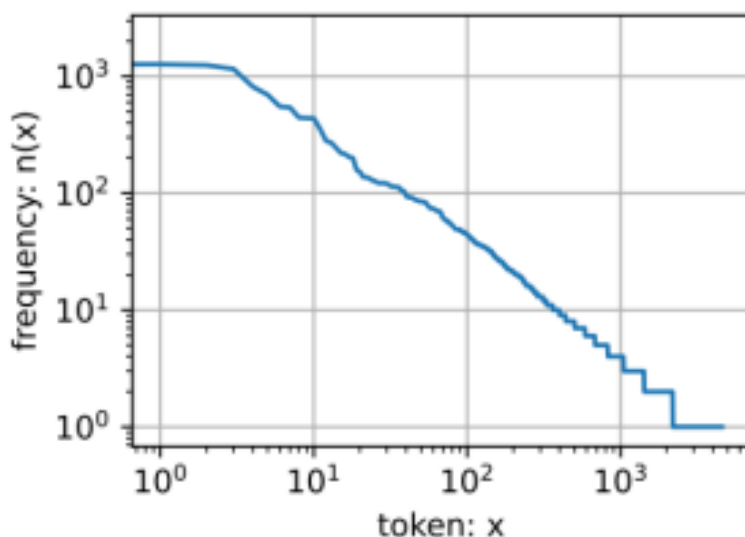
$$P(x_1, x_2, x_3, x_4) = P(x_1)P(x_2)P(x_3)P(x_4),$$

$$P(x_1, x_2, x_3, x_4) = P(x_1)P(x_2 | x_1)P(x_3 | x_2)P(x_4 | x_3),$$

$$P(x_1, x_2, x_3, x_4) = P(x_1)P(x_2 | x_1)P(x_3 | x_1, x_2)P(x_4 | x_2, x_3).$$

停用词：很常用但是没什么实际含义的词。例如英语中的 "a" "the" 等

齐普夫定律 (Zipf's law)：在对数坐标下，一段自然文本中单词的出现频率和其“按照出现频率的排名”基本上呈现线性关系，如下图所示



启示：少数词（尤其是停用词）出现极其频繁，而大多数词出现很少

这进一步说明：**如果只靠统计词频，低频词和长词组会非常难建模。**

长序列的读取：分段

一整个序列很长，无法直接输入到模型中，此时需要对其进行分段。

注意：目前的序列都是 token 排列而成的，分段不会破坏 token。

分段时，取**两个**固定长的小序列 (X, Y) ，其中 Y 是 X 向后移一位得到的

为什么这么做？让模型学习前后对应关系（对应位置的 X, Y 一一对应得到）。

具体理解见下例子

有两种分段方式：

1. **随机采样：**从长序列里随机挑一些片段组成小批量
2. **顺序分区：**按照原始顺序挨个分区

举例：假设原始序列是 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, ...

随机采样：第一次, $X = [3, 4, 5, 6, 7]$ 则 $Y = [4, 5, 6, 7, 8]$

由此，模型学习：3 的下一个是 4, 4 的下一个是 5,

第二次: $X = [1, 2, 3, 4, 5]$ 则 $Y = [2, 3, 4, 5, 6]$

顺序分区：第一次 $X = [0, 1, 2, 3, 4]$, $Y = [1, 2, 3, 4, 5]$

第二次 $X = [5, 6, 7, 8, 9]$, $Y = [6, 7, 8, 9, 10]$

区别：随机采样随机性强，顺序分区能够保证来自两个相邻的小批量中的子序列在原始序列上也是相邻的。

循环神经网络

先回忆一下最基础的多层感知机：数据输入，经过线性变换和隐藏层非线性变换直接得到输出，两次数据的输入之间没有关联。

其公式是

$$H = \phi(XW_{xh} + b_h)$$

也就是说，输出只看当前输入 X （这里的输出就是 H ）

但是，语言模型等要更加复杂，输出要结合从最初到当前的所有信息。

考虑一开始的隐变量自回归模型： t 时刻数据输入，结合前 $t - 1$ 个输入得到的 h_{t-1} ，生成新的，前 t 个输入的总结 h_t （这一步生成其实就是用了多层感知机），然后根据这个新的总结 h_t 经过线性变换得到输出 O_t （注意这里不需要非线性变换了），即

$$H_t = \phi(X_t W_{xh} + H_{t-1} W_{hh} + b_h)$$

$$O_t = H_t W_{hq} + b_q$$

注意：一共有五个要学习的参数：**两个偏置 (b)**，**三个系数 (W)**

（当然这五个都是 tensor，也就是说真正的数字量还由输入输出和中间 tensor 的形状决定）

这五个系数都是和 t 无关的，也就是说不管要预测哪个时刻都用这同一套参数

因此，循环神经网络的参数开销不会随着时间步的增加而增加。

训练时，RNN 把划分出的 X 作为输入，在每个时间步预测下一个词元，得到预测 $\hat{Y}$ ，再和真实标签 Y 计算损失，通过时间反向传播来更新 $W_{xh}, W_{hh}, b_h, W_{hq}, b_q$ 这 5 组参数；预测时，模型根据给定输入或前缀输出下一个词元的概率分布，并可以一步步生成后续序列。

困惑度 (Perplexity, PPL)

顾名思义，是一个衡量模型在预测时的“困惑”程度的量，其公式为

$$\exp \left(-\frac{1}{n} \sum_{t=1}^n \log P(x_t \mid x_{t-1}, \dots, x_1) \right)$$

理解：实际上，其核心内容是以下公式：

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_{t-1}, \dots, x_1).$$

其中， x_t 是第 t 个**正确的** token。 $P(x_t \mid x_{t-1}, \dots, x_1)$ 的含义就是：在已知前 $t - 1$ 个 token 的情况下，第 t 个 token 预测正确的概率。

也就是说，困惑度是衡量模型预测**一整段话**时的标准。这一整段话有 n 个 token

然后进行取对数把乘法变成加法，再取平均数（让不同长度的序列可比）。

更直观理解：困惑度公式可以变形为

$$\left(\prod_{t=1}^n \frac{1}{P(x_t \mid x_{t-1}, \dots, x_1)} \right)^{1/n}$$

也就是说

困惑度 = 真实词元概率倒数的几何平均。

如果模型每一步都给真实答案概率 p ，那么：

$$\text{PPL} = \frac{1}{p}$$

这就是困惑度最直观的含义：困惑度大约表示：模型平均每一步像是在多少个**等可能**选项中猜出正确答案。

三种极端情况：

- 困惑度 = 1 最好，完全不困惑（预测结果百分百正确）
- 困惑度 = 词表大小 在整个词表里面随机猜
- 困惑度 = ∞ 最差，真实答案概率为 0（完全得不到正确结果）

总而言之，困惑度就是模型平均每预测一个词元时，相当于在多少个等可能选项中犹豫。

困惑度越低，语言模型越好。

循环神经网络的具体实现

（主要理解实现中的一些操作细节，代码部分略去）

1. 我们给循环神经网络输入的是 token，但是**不代表我们输入的是一个数字**（比如 token 的那个数字）。为什么？因为具体的词和词之间不存在数量上的加减和距离关系。比如 token 为 1, 2, 3 的三个词，说 $3 = 1 + 2$ 是没有意义的，说 1 和 2 的距离比 1 和 3 之间的距离近也是无意义的。事实上，我们有下面两种方案：

独热编码 (onehot)：词表中有 N 个不同的 token，就用 $1 \times N$ 的 tensor 来表示每一个 token，每个 token 都是**正交的单位向量**。

例如：有三个词，token 分别为 1, 2, 3

则这三个词对应的独热编码为 $[1, 0, 0]$, $[0, 1, 0]$, $[0, 0, 1]$

由于完全正交，词与词之间不存在任何关联。

优点：确定，无需训练，不会引入错误的关系

缺点：维度大占空间，词和词之间没有语义关系

词嵌入 (embedding)：把每个 token 映射为一个相对 onehot 更短的向量，例如 100 个词，每个词用一个 1×4 的向量来表示。比如：

cat $\rightarrow [0.21, -0.38, 0.72, 0.11]$

dog $\rightarrow [0.18, -0.41, 0.69, 0.08]$

banana $\rightarrow [-0.55, 0.13, -0.22, 0.91]$

注意：这里的向量不是人工规定的，而是**模型训练出来的参数**。

如果训练得好，**语义相近的词，它们的向量也会更接近**：比如上面的 cat 和 dog 就比较接近，而和 banana 相距较远（指四维点之间的坐标）

2. 我们的模型训练好之后，给定一段文字让其预测后面的文字时，要先经过**预热 (warm-up) 期**：该阶段不改变训练的结果，只是根据给定的字符串**计算出最后一个，要参与预测下一字符的隐状态**（未经过预热时，隐状态一般重置为 0）

3. **梯度裁剪**：为了避免反向传播中的**梯度爆炸**（注意**不能避免梯度消失**），采用梯度裁剪。

具体操作：给梯度的**长度**设一个**上限**。当梯度太大时，把它按比例缩短；当梯度不大时，保持不变。它可以**防止** RNN 训练时因为梯度爆炸导致**参数更新过猛**，从而提升训练稳定性。

4. 训练时，需要优化的损失是**交叉熵损失**，而模型效果指标是**困惑度** perplexity

（其实两者没啥区别，困惑度 = $\exp(\text{交叉熵损失})$ ）

5. 反向传播时，RNN 和之前的其他神经网络有一点不同：由于隐状态存在时间上的一层层依赖，导致随着预测时间的增加，梯度**反向传播链条会越来越长**，没有上界。这一现象称为**通过时间反向传播 (Backpropagation Through Time, BPTT)**

这会导致难以训练。并且这也会导致梯度爆炸和梯度消失。

举例：假设前向传播时依据以下的简化式子：

$$h_t = W_{hx}x_t + W_{hh}h_{t-1}$$

反向传播时，隐状态梯度会反复乘上 $W_{hh}^\top$ 。

也就是说，会出现类似：

$$(W_{hh}^\top)^k$$

这样的矩阵高次幂。

这就是问题的根源：

如果矩阵反复相乘后越来越大 → 梯度爆炸

如果矩阵反复相乘后越来越小 → 梯度消失

即：**矩阵高次幂会导致特征值发散或消失，表现为梯度爆炸或梯度消失。**

如何处理这一问题？很自然的想法是，如果把太久远之前的连接**截断**，只看最近的连接，就可以把反向传播链条的**长度限定**在一定范围内，从而能够正常训练。

有两种截断方法：**常规截断**（每一段的长度固定），**随机截断**（每一段的长度随机）

具体反向传播细节：

假设采用常规截断，截断长度为 35，也即：

第 1 段: $1, 2, 3, \dots, 35$

第 2 段: $36, 37, 38, \dots, 70$

第 3 段: $71, 72, 73, \dots, 105$

首先, 截断不影响正向传播中隐状态的传播。例如 h_{35} 和 x_{36} 依旧能通过变换得到 h_{36}

截断影响的是反向传播中追溯的上限: 某个位置的反向传播的链条最远只能追溯到该段的段首, 不能再往前追溯。截断 BPTT 中通常不是对某个单独 token 单独反向传播, 而是对当前片段的整体交叉熵损失做一次反向传播; 梯度只在当前片段内传播, 然后用这些梯度更新 RNN 的全局共享参数。

粗糙一点理解: 就是反向传播时各个片段分开, 每个片段训练自己的参数, 更近的片段不会传播到更远的片段。